

ANDROID INTERN ASSIGNMENT

Movie Explorer App

Official Technical Assignment - Android Internship Program

Issued By	Mobile App Development Team
Assignment Type	Individual Android Development Task
Recommended Duration	7 calendar days from the assignment date
Platform	Android - Kotlin + XML
API	The Movie Database (TMDB) API

Purpose

Build a small, production-style Android application that demonstrates core Android fundamentals, API integration, local storage, state handling, and disciplined Git usage. The task is intentionally limited in scope so the focus remains on learning, code quality, and development workflow.

Important: Complete the required features first. Bonus features are optional and should only be attempted after the mandatory scope is stable.

1. Assignment Overview

Create an Android application named Movie Explorer using Kotlin and XML. The app will load movie data from TMDb, allow users to search movies, view details, and save favourites locally. The implementation should follow a simple MVVM structure and use Git throughout development.

Learning Outcomes

- Consume a REST API using Retrofit and Coroutines.
- Build screens with XML, RecyclerView, and Navigation Component.
- Manage UI state using ViewModel and LiveData or StateFlow.
- Persist favourite movies with Room Database.
- Use Dependency Injection with Hilt or Koin.
- Handle loading, empty, error, and offline states.
- Use feature branches, meaningful commits, and Pull Requests.

Required Technology

Language & UI	Kotlin, XML, RecyclerView, ViewBinding
Architecture	MVVM, Repository Pattern
Data	Retrofit, OkHttp, Coroutines, Room
App Structure	Navigation Component, ViewModel, Hilt or Koin

2. Required App Screens

Screen	Minimum Requirement
Home	Show popular movies in a RecyclerView. Each item should include poster, title, release date, and rating.
Movie Details	Show poster/backdrop, title, overview, release date, rating, genres, and favourite action.
Search	Search movies by title and display results. Include loading, empty, and error states.
Favourites	Display movies saved locally in Room. Users must be able to remove favourites and access this screen offline.

API Endpoints

- GET /3/movie/popular
- GET /3/search/movie
- GET /3/movie/{movie_id}

3. UI Reference

Use the following screens as a visual direction only. Exact duplication is not required; the final UI should remain clean, readable, and consistent.

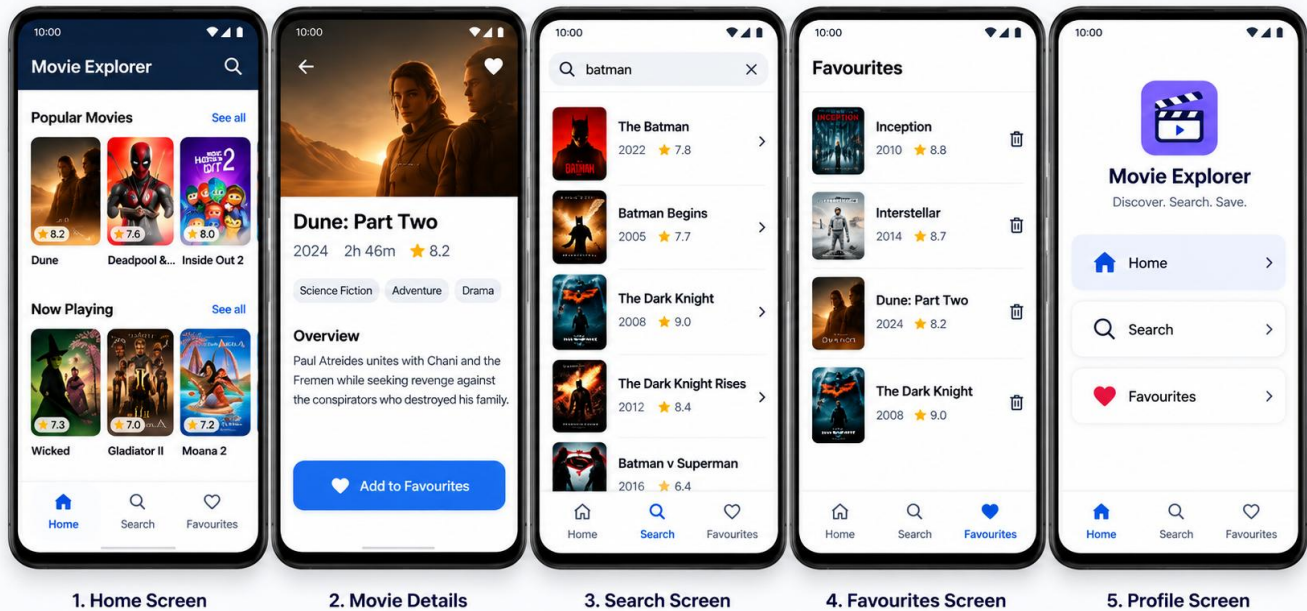


Figure 1. Suggested mobile screen flow for Movie Explorer

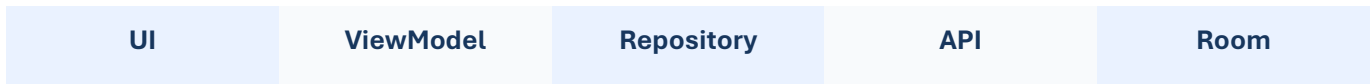
UI Expectations

- Consistent spacing and typography.
- Responsive layouts for common phone sizes.
- Clear loading, empty, error, and retry states.
- No hardcoded user-facing text when resources should be used.
- Movie posters should load through Glide or Coil.

4. Technical Rules

- Retrofit must not be called directly from an Activity or Fragment.
- Room operations must not be performed directly from an Activity or Fragment.
- Use ViewModel to expose UI state.
- Use a Repository to coordinate remote and local data.
- Use Hilt or Koin for dependency injection.
- Do not hardcode the TMDB API key or access token in committed source code.
- The application must not crash because of a network or API failure.

Suggested Simple Flow



5. Git Workflow

Git usage is part of the evaluation. The repository history should show how the project was built, not only the final code.

Required Branches

- main - stable/final code
- develop - integration branch
- feature/home
- feature/details
- feature/search
- feature/favourites

Commit Examples

- chore: initialize android project
- feat: add tmdb movie list
- feat: implement movie details screen
- feat: add search feature
- feat: add room favourites
- fix: handle empty search result
- docs: update readme

Pull Request Rule

- Complete work in a feature branch and open a Pull Request into develop.
- Do not implement the whole project directly on main.
- PR description should briefly explain what was added or changed.

6. Recommended 7-Day Plan

Day	Focus	Expected Output
Day 1	Setup	Project, Git branches, TMDB configuration, Retrofit, base navigation
Day 2	Home	Popular movie list and basic UI states
Day 3	Details	Movie details screen and navigation
Day 4	Search	Movie search with empty/error/loading states
Day 5	Favourites	Room database and favourite add/remove
Day 6	Quality	Error handling, refactoring, offline checks, README
Day 7	Delivery	Final testing, PR review, APK and submission

8. Evaluation Criteria

Area	What We Review
Functionality	Required screens and features work correctly.
Android Fundamentals	Correct use of ViewModel, RecyclerView, lifecycle, and navigation.
API & Data	Retrofit, Room, repository flow, and error handling are implemented properly.
Code Quality	Readable structure, naming, low duplication, maintainable code.
Git Usage	Feature branches, meaningful commits, and Pull Requests.
UI/UX	Clean, consistent, usable screens and proper application states.
Documentation	README is sufficient for another developer to run the project.

Review Note: The intern should be able to explain the submitted code, architecture, API flow, Room usage, and Git history during the review.